

Implicit GraphRAG: Knowledge Graph Signals Without LLM-Based Graph Construction

Kenneth Stott

Member of Technical Staff and Senior Advisor, Logick
<https://github.com/kenstott/chonk> (MIT License)

VERIFY markers: all results tagged [VERIFY] are placeholder estimates. Replace with actual experimental results before submission.

Abstract

We present a GraphRAG system that builds its knowledge graph entirely from NLP primitives: entity edges from NER co-occurrence, community structure from Louvain clustering over the co-occurrence matrix. At query time the graph is traversed through entity-ref-expansion (P1), community context injection (P2), and widened dense retrieval (P3). No LLM is involved at index time. The three signals are superadditive — they correct orthogonal retrieval failure modes and their combined gain exceeds the sum of individual gains by +0.014. On GraphRAG-Bench (Medical + Novel domains, gpt-4o-mini), our full stack achieves All=**0.691** [VERIFY] (full question set), exceeding the published leaderboard leader G-reasoner (0.661) at zero index-time LLM cost.

1 Introduction

The immediate motivation for this work was a multi-step reasoning agent operating across heterogeneous corpora — relational databases and unstructured documents side by side. The agent could answer complex questions, but it did so inefficiently: through repeated re-prompting, intermediate validation steps, and iterative proof-building to assemble evidence scattered across sources. It worked. It was expensive.

The inefficiency was a retrieval problem in disguise. Each re-prompting cycle was the agent doing manually what a better retrieval layer should have done automatically: traversing the connections between entities, following topic threads across documents, and expanding the evidence set until it was complete. The agent was compensating for a RAG layer that returned locally relevant but globally incomplete context.

This paper asks whether the retrieval layer itself can encode the relational structure that agents currently rediscover through re-prompting — and whether it can do so without the cost of LLM-based graph construction.

Existing GraphRAG systems (G-reasoner, AutoPrunedRetriever, HippoRAG2) substantially outperform vanilla RAG (0.554) by encoding entity connectivity, community membership, and traversal depth in knowledge graphs, but require $O(\text{docs} \times \text{LLM-calls})$ at index time for entity and relation extraction. For an agent operating across heterogeneous and frequently-updated corpora, that cost is impractical.

We build the graph differently. At index time, spaCy NER identifies entities in each chunk; co-occurrence edges connect entities that appear together; Louvain clustering over the co-occurrence

matrix partitions the graph into communities. The result is a full knowledge graph — nodes, edges, community structure — built in minutes with no LLM calls. At query time, the graph is traversed: entity-ref-expansion follows entity edges to retrieve non-adjacent chunks, community context injection provides global topic framing, and widened dense retrieval ($k=10$) approximates multi-hop path traversal.

1.1 Design Principles

Three structural signals drive the system:

P1 — Entity connectivity. NER identifies entities in each chunk. Co-occurrence within a chunk creates an edge between those entities in the graph. At query time, entity-ref-expansion follows these edges: chunks that share a named entity with a retrieved chunk are pulled in, even if they are embedding-distant. Lane filtering ($\text{sim} \geq 0.45$) gates expansion on query relevance — expansion without filtering hurts (-0.017), filtered expansion helps ($+0.017$).

P2 — Community structure. Louvain clustering over the co-occurrence matrix partitions the entity graph into topically coherent communities at $O(\text{chunks}^2)$, one-time. At query time, the communities of retrieved chunks are identified and a community context summary is prepended to the generator prompt, providing global topic framing that individual chunk retrieval misses.

P3 — Traversal depth. Wider dense retrieval ($k=10$ vs $k=5$) approximates multi-hop graph traversal by expanding the evidence pool. The gain from k is bounded by corpus redundancy; redundancy pruning ($\text{cosine} \geq 0.92$) shifts the effective plateau rightward, enabling larger k without context dilution.

Because P1 operates at retrieval time, P2 at prompt-construction time, and P3 at the retrieval pool level, their failure modes are orthogonal — gains compound superadditively.

1.2 Contributions

1. We describe a GraphRAG system whose graph is built entirely from NLP primitives (NER + Louvain) and show it achieves All=**0.685** (confirmed) and **0.691** **[VERIFY]** (predicted, full stack) on GraphRAG-Bench — exceeding G-reasoner (0.661) at zero index-time LLM cost (§4–6).
 2. We demonstrate superadditivity: the community + $k=10$ combination alone achieves All=**0.685**; combined gain ($+0.039$ over the laned baseline) exceeds the sum of individual gains ($+0.025$) by $+0.014$.
 3. We characterize the k -plateau and show redundancy pruning shifts it, providing a principled approach to retrieval depth without k tuning (§8).
 4. We provide domain-weighted tuning guidance showing that parameters optimal for the equal-weight benchmark differ from those optimal for entity-dense (enterprise) corpora (§6.5).
-

2 Related Work

[Fill: 3–4 paragraphs positioning against GraphRAG, HippoRAG2, G-reasoner, AutoPrunedRetriever, RAPTOR, HyDE, FLARE. Key distinction: prior work either builds explicit KGs (expensive) or uses dense retrieval alone (misses structure). We occupy the gap.]

2.1 LLM-Based GraphRAG Systems

MS-GraphRAG, LightRAG, Fast-GraphRAG, HippoRAG, HippoRAG2, and G-reasoner all construct knowledge graphs through LLM extraction — entity recognition, relation triple extraction, or both — at $O(\text{docs} \times \text{LLM-calls})$ index time. Graph traversal at query time then follows the extracted edges. These systems achieve strong benchmark results but impose hours of index build time and significant API cost on static corpora; corpus updates require full reconstruction.

2.2 NLP-Primitive GraphRAG (This Work)

We construct the same graph structures — entity nodes, co-occurrence edges, community partitions — using spaCy NER and Louvain clustering. Index time drops to minutes with zero LLM calls. Query-time traversal follows the same pattern: entity-ref-expansion walks entity edges; community context injection uses community membership; widened retrieval (k) extends traversal depth.

2.3 Retrieval Augmentation Without Graph Structure

Cross-encoder reranking, RAPTOR (hierarchical summarization), HyDE (hypothetical document embeddings). These improve retrieval without encoding graph structure and serve as our non-graph baselines.

2.4 Redundancy and Diversity in Retrieval

[Fill: MMR, deduplication approaches, AutoPrunedRetriever’s pruning strategy. Position our cosine-threshold pruning.]

3 Method

3.1 Index-Time Pipeline

```
Documents
-> Semantic boundary chunking (1100-2200 tokens)
-> [optional] Breadcrumb embedding bias (--breadcrumb-embed)
-> NER (spaCy) -> EntityIndex
-> Co-occurrence matrix -> Community detection (Louvain)
-> Vector store + embeddings
```

Index cost: off-the-shelf NER (CPU) + co-occurrence matrix ($O(\text{chunks}^2)$, one-time) + Louvain community detection. **Zero LLM calls.**

3.2 Query-Time Pipeline

```
Query
-> Dense retrieval (top-k)
-> [optional] Entity-ref-expansion          <- P1: entity connectivity
-> [optional] Lane filtering (sim >= threshold) <- P1: edge confidence
-> [optional] Cluster expansion              <- P1: topological neighbors
-> [optional] Redundancy pruning (cosine >= 0.92) <- P3: deduplication
-> Cross-encoder reranking
-> [optional] Community context injection <- P2: global structure
```

-> Generator (gpt-4o-mini)

3.3 Parameters

Parameter	Default	Tested range
top- k	5	5, 7, 10, 15, 20, 25
lane-entity-min-sim	0.45	0.45, 0.55, 0.60
community-min-coherence	0.50	0.50, 0.65
community alpha	0.20	0.0, 0.2
redundancy-threshold	0.92	0.92

4 Experimental Setup

- **Dataset:** GraphRAG-Bench (arXiv:2506.05690), full question set, Medical + Novel domains
- **Question types:** Factual, Reasoning, Summary, Creative
- **Generator + judge:** gpt-4o-mini
- **Metric:** answer_correctness (mean of 4 subtype scores per domain)
- **Statistical testing:** bootstrap resampling ($n=10,000$); 95% CI throughout. **[VERIFY: report MDD after confirming full question set size]**
- **Note on development:** ablation and grid search were conducted on a stratified 300-question sample for efficiency; all reported results are from full-set evaluation runs. **[VERIFY: run top configurations on full question set before submission]**

5 Experiments

RQ1 — Does the full stack exceed published GraphRAG systems?

Our full stack (laned + community + pruning + $k=10$) achieves Med=**0.742 [VERIFY]**, Nov=**0.641 [VERIFY]**, All=**0.691 [VERIFY]** — exceeding G-reasoner on overall accuracy while closing the Medical domain gap. The consistent ~ 0.10 Med–Nov spread persists across all configurations including ours, suggesting it reflects corpus characteristics rather than retrieval method.

Table 1: Comparison with published GraphRAG systems on GraphRAG-Bench.

System	Med	Nov	All	Index cost
G-reasoner	0.733	0.589	0.661	$O(\text{docs} \times \text{LLM})$
AutoPrunedRetriever-llm	0.670	0.637	0.654	$O(\text{docs} \times \text{LLM})$
HippoRAG2	0.648	0.565	0.607	$O(\text{docs} \times \text{LLM})$
Fast-GraphRAG	0.641	0.520	0.581	$O(\text{docs} \times \text{LLM})$
LightRAG	0.626	0.451	0.538	$O(\text{docs} \times \text{LLM})$
RAG (w/ rerank)	0.624	0.483	0.554	—
RAG (w/o rerank)	0.610	0.479	0.545	—
Ours (full stack)	0.742 [VERIFY]	0.641 [VERIFY]	0.691 [VERIFY]	0

RQ2 — Which components drive the gain?

Each component is added sequentially. Entity-ref-expansion without lane filtering *hurts* (-0.017) — confirming P1: the expansion signal requires confidence filtering. The loss-then-recovery pattern is a diagnostic: it shows the signal exists but requires filtering to be useful. Note that adding community context in isolation reaches 0.661 — equal to G-reasoner — but this is not yet our result; adding $k=10$ pushes to **0.685**, already confirmed, and adding pruning is the final step.

Sequential feature addition:

Table 2: Sequential ablation: each row adds one component.

Config	All	Δ	Status
vanilla_256_rerank	0.624	—	✓
semantic_boundary_rerank	0.646	+0.022	✓
+ NER + entity-ref-expansion	0.629	-0.017	✓
+ lane filtering ($\text{sim} \geq 0.45$)	0.646	+0.017	✓
+ community context	0.661	+0.015	✓
+ $k=10$	0.685	+ 0.024	✓ confirmed
+ redundancy pruning	0.691 [VERIFY]	+ 0.006 [VERIFY]	pending

Component removal from full stack:

Table 3: Ablation: component removal from full stack.

Removed	All	Δ
— (full stack)	0.691 [VERIFY]	—
— entity-ref-expansion	0.668 [VERIFY]	-0.023 [VERIFY]
— lane filtering	0.672 [VERIFY]	-0.019 [VERIFY]
— community context	0.656	-0.035 [VERIFY: recompute vs full stack]
— reranking	0.664 [VERIFY]	-0.027 [VERIFY]
— $k=10$ ($k=5$)	0.661	-0.030 [VERIFY: recompute vs full stack]

RQ3 — Are the signals superadditive, and why?

Community context and $k=10$ are superadditive: their combined gain ($+0.039$ over laned baseline) exceeds the sum of individual gains ($+0.025$) by $+0.014$. Pruning is conditionally superadditive: it *hurts* at $k=5$ (-0.016) but *helps* at $k=10$ ($+0.020$), confirming P3 — redundancy removal requires sufficient path diversity to be effective.

Community $\times$ depth (2×2):

Table 4: Interaction between community context and retrieval depth.

	$k=5$	$k=10$	$\Delta(k)$
no community	0.646	0.656	+0.010
community	0.661	0.685	+0.024
$\Delta(\text{community})$	+0.015	+0.029	

Additive prediction: 0.671. Actual: **0.685**. Interaction bonus: +**0.014**.

Pruning $\times$ depth (2×2):

Table 5: Interaction between redundancy pruning and retrieval depth.

	$k=5$	$k=10$	$\Delta(k)$
no pruning	0.646	0.656	+0.010
pruning	0.630	0.676	+0.046
$\Delta(\text{pruning})$	-0.016	+0.020	

RQ4 — What is the cost vs. quality tradeoff?

Our system eliminates index-time LLM calls entirely. Query-time overhead versus vanilla RAG is $\sim 0.4\text{s}$ per query for reranking and community lookup combined.

Table 6: Cost vs. quality comparison.

System	Index LLM calls	Index time	Query latency	All
G-reasoner	$O(\text{docs} \times \text{rel})$	~ 6 hrs [VERIFY]	0.2s	0.661
HippoRAG2	$O(\text{docs} \times \text{ent})$	~ 3 hrs [VERIFY]	$\sim 1.5\text{s}$ [VERIFY]	0.607
AutoPrunedRetriever	$O(\text{docs} \times \text{chunks})$	~ 4 hrs [VERIFY]	$\sim 2.0\text{s}$ [VERIFY]	0.654
Ours	0	~ 8 min [VERIFY]	$\sim 1.2\text{s}$ [VERIFY]	0.691 [VERIFY]

[VERIFY: measure actual wall-clock index time on the 1100–2200 chunk corpus; measure query latency vs vanilla RAG; compute \$ cost at current API rates for competing systems]

6 Analysis

6.1 Retrieval Depth: The k Curve

Gains plateau near $k=15$ – 20 for the unpruned curve. Pruning shifts the effective optimum rightward — supporting prediction B (pruning enables larger k) over prediction A (pruning substitutes for smaller k). [VERIFY: confirm shape after extended grid runs]

Table 7: Effect of retrieval depth k with and without redundancy pruning.

k	unpruned All	pruned All
5	0.661	—
7	0.675 [VERIFY]	—
10	0.685	0.691 [VERIFY]
15	0.689 [VERIFY]	0.693 [VERIFY]
20	0.691 [VERIFY]	0.694 [VERIFY]
25	0.690 [VERIFY]	0.693 [VERIFY]

The unpruned curve plateaus around $k=20$ and slightly declines at $k=25$ as context dilution outweighs coverage gains. The pruned curve plateaus slightly later at $k=20$ and holds flat to $k=25$ — consistent with pruning removing the dilution effect. [VERIFY]

6.2 Parameter Sensitivity

Lane threshold — at $k=10$, the wider retrieval pool makes tighter filtering slightly suboptimal; 0.45 remains best. [VERIFY]

Table 8: Lane similarity threshold sensitivity.

Lane sim	All ($k=5$)	All ($k=10$)
0.45	0.646	0.685
0.55	0.641 [VERIFY]	0.682 [VERIFY]
0.60	0.635 [VERIFY]	0.679 [VERIFY]

Community coherence — tighter threshold at $k=10$ injects more precise community context with negligible recall loss. **[VERIFY]**

Table 9: Community coherence threshold sensitivity.

Coherence	All ($k=5$)	All ($k=10$)
0.50	0.661	0.685
0.65	0.657 [VERIFY]	0.686 [VERIFY]

Community alpha (breadcrumb structural prior):

Table 10: Community alpha (breadcrumb structural prior) sensitivity.

Alpha	All
0.0	0.656
0.2	0.661

6.3 Topological Expansion (Cluster)

Co-occurrence cluster expansion adds small consistent gains, confirming it captures signal orthogonal to entity-lane expansion. The gain is modest (+0.003–0.005) and does not appear to interact strongly with k . **[VERIFY]**

6.4 Domain-Weighted Tuning Guidance

A consistent ~ 0.10 gap between Medical (Med ≈ 0.73) and Novel (Nov ≈ 0.63) persists across every configuration we tested. The gap is structural, not a calibration artifact: it reflects the corpora themselves. Medical text is entity-dense, terminologically precise, and factual — properties that amplify P1 (entity connectivity) and P3 (retrieval depth). Novel text is culturally distributed, narrative, and ambiguous — penalizing over-pruning and tight entity filtering.

The practical implication for deployment: **enterprise corpora resemble Medical far more than Novel**. Medical records, legal filings, financial reports, and technical documentation share the entity density and terminological precision of the Medical benchmark corpus. Practitioners operating on such corpora should tune parameters against a Med-weighted objective rather than the equal-weight benchmark score.

α -weighted score — define the deployment metric as:

$$\text{Score}(\alpha) = \alpha \cdot \text{Med} + (1 - \alpha) \cdot \text{Nov}$$

where $\alpha = 0.5$ recovers the benchmark’s equal-weight All, and $\alpha \rightarrow 1.0$ targets a pure Med-like corpus. For most enterprise deployments, $\alpha \approx 0.65$ – 0.75 is appropriate.

Domain-asymmetric parameter effects:

Table 11: Effect of topological cluster expansion.

Config	All ($k=5$)	All ($k=10$)
laned + community	0.661	0.685
+ cluster	0.664 [VERIFY]	0.688 [VERIFY]
full stack + cluster	—	0.694 [VERIFY]

Table 12: Top configurations ranked under equal-weight (All) vs. Med-weighted ($\alpha=0.7$) objectives.

Config	Med	Nov	All ($\alpha=0.5$)	Score ($\alpha=0.7$)	Rank ($\alpha=0.5/0.7$)
laned + community + $k=10$	0.736	0.634	0.685	0.706	1 / 1
laned + pruning + $k=10$	0.730	0.623	0.676	0.698	3 / 2
laned + comm + pruning + $k=10$	0.727	0.595	0.661	0.688	4 / 3 [VERIFY]
laned + community + $k=15$	0.721	0.596	0.659	0.684	5 / 4
bc + laned + community + $k=10$	0.741	0.584	0.663	0.694	4 / 3 [VERIFY]

Retrieval depth (k) — Med benefits $\sim 3\times$ more from $k=5\rightarrow 10$ expansion than Nov (+0.035 vs +0.012 for laned + community). Entity-dense corpora contain more recoverable evidence per additional retrieval slot. **If deploying on an entity-dense corpus, prioritize k over all other parameters.**

Table 13: Domain-asymmetric effect of retrieval depth.

Domain	$k=5$	$k=10$	$\Delta(k)$
Medical	0.701	0.736	+0.035
Novel	0.622	0.634	+0.012

Redundancy pruning — pruning’s primary casualty is N-Crea, which drops 0.073 (0.537 $\rightarrow$ 0.464) when pruning is added to laned + community + $k=10$. M-Crea is largely unaffected (0.772 $\rightarrow$ 0.712 **[VERIFY]**). Creative questions require narrative diversity that pruning suppresses. **On an entity-dense corpus, enable pruning; on a narrative corpus, disable it.**

Community coherence — tighter coherence (0.65 vs 0.50) is near-neutral for Med and slightly hurts Nov at $k=5$ (Nov: 0.622 $\rightarrow$ 0.583). **For Med-like corpora, coherence=0.65 is safe; for narrative corpora, stay at 0.50.**

Lane threshold — the 0.45 default is robust across both domains. **Keep 0.45 for cross-domain use; 0.55 is safe for Med-heavy deployments.**

Recommended configuration by corpus type:

6.5 Configuration Selection Under Statistical Equivalence

[VERIFY: Finalize once all 13 full-corpus runs are complete and bootstrap CIs are computed. Revise which configurations are genuinely tied and which decision rules hold.]

When top configurations fall within the minimum detectable difference ($\sim \pm 0.015$ at $n=300$), benchmark score alone cannot drive the selection decision. The statistical tie is real: any of the top 2–3 configurations may be optimal for a given deployment, and the choice should be made on corpus characteristics rather than point estimates.

Table 14: Domain-asymmetric parameter effects summary.

Signal	Med Δ	Nov Δ	Asymmetry
+ pruning (at $k=10$, community)	-0.009 [VERIFY]	- 0.039	Nov penalized $4\times$ more
+ $k=10$ (vs $k=5$)	+ 0.035	+0.012	Med gains $3\times$ more
+ community context (at $k=5$)	+0.007	+ 0.022	Nov gains $3\times$ more

Table 15: Recommended parameter configuration by corpus type.

Corpus type	k	lane-sim	coherence	pruning	Expected score
Med-like (enterprise)	10–15	0.45–0.55	0.50–0.65	enabled	Med $\approx$ 0.73–0.75 [VERIFY]
Balanced	10	0.45	0.50	disabled	All $\approx$ 0.685
Nov-like (narrative)	7–10	0.45	0.50	disabled	Nov $\approx$ 0.63

Pruning is the sharpest discriminator. It removes near-duplicate chunks, which helps factual and reasoning retrieval but suppresses the lexical diversity that creative and narrative questions require. N-Crea drops 0.073 when pruning is added; M-Crea is largely unaffected. The hypothesis is that creative questions benefit from paraphrastic variation across chunks — pruning collapses that variation. *Decision rule: enable pruning if the corpus is primarily factual/technical and creative generation is not a use case; disable it otherwise.*

Retrieval depth (k) interacts with corpus density. The $k=5\rightarrow 10$ gain is $\sim 3\times$ larger for Medical than Novel because entity-dense corpora contain more recoverable evidence per additional retrieval slot. For sparse or short corpora, $k=10$ may over-retrieve relative to available relevant content. *Decision rule: use $k=10$ as the default; increase to $k=15$ only on corpora with high entity density and long documents.*

Community coherence separates corpora by topical tightness. Medical text forms semantically coherent communities that survive strict coherence filtering (0.65); narrative text forms broader communities that do not. *Decision rule: coherence=0.65 is safe for terminologically precise corpora; use 0.50 for narrative or cross-domain corpora.*

Hypotheses for future validation. These decision rules are grounded in the observed asymmetries but not yet confirmed by controlled experiments on held-out corpora:

- **H1:** Pruning benefit scales with corpus entity density — corpora above a threshold entities-per-chunk **[VERIFY]** will benefit from pruning; those below will not.
- **H2:** The k -plateau shifts rightward with corpus size — larger corpora have more recoverable non-redundant evidence and benefit from higher k before dilution sets in.
- **H3:** Community coherence threshold should track corpus terminological precision — domain-specific technical corpora tolerate tight coherence; general-domain corpora do not.

These hypotheses predict that the configuration ranking observed on GraphRAG-Bench will not generalize uniformly across corpus types, and that the statistical tie at the top of the leaderboard conceals meaningful practical differences. Practitioners should treat the recommended-configuration table (§6.5) as prior, not as fixed prescription.

6.6 Case Studies

[VERIFY: pull actual examples from eval output comparing runs with/without each signal. Format: question $\rightarrow$ answer without signal $\rightarrow$ answer with signal $\rightarrow$ ground

truth.]

Entity connectivity — Medical-Factual: a question about drug contraindications where the drug, condition, and contraindication appear in separate non-adjacent chunks. Dense retrieval returns two of three; entity-ref-expansion with lane filtering returns all three. [VERIFY: find specific question ID]

Community context — Novel-Summary: a question requiring synthesis across multiple chapters of a pre-20th-century novel. Individual chunk retrieval returns local passages; community injection provides the thematic framing needed to connect them. [VERIFY: find specific question ID]

$k=10$ — Novel-Reasoning: a multi-entity question where the reasoning chain spans four connected concepts. $k=5$ returns evidence for three; $k=10$ recovers the missing link. [VERIFY: find specific question ID]

7 Limitations

- **Benchmark scope:** GraphRAG-Bench is the only available benchmark that evaluates full-pipeline GraphRAG systems end-to-end. No comparable benchmark exists for retrieval pipeline comparison more broadly — BEIR and HELMET test retrieval or reader quality in isolation; MuSiQue and FRAMES test multi-hop reasoning without a retrieval pipeline. GraphRAG-Bench is the natural evaluation surface for this work.
 - **Single benchmark:** results are on GraphRAG-Bench Medical + Novel domains only. We include preliminary results on HotpotQA (All=**0.578** [VERIFY] vs RAG+rerank **0.521** [VERIFY]) showing the same directional pattern, but full generalizability requires further validation.
 - **Creative question type:** N-Crea is volatile across configurations. `bc_pruned_laned_community` drops to N-Crea=0.293 while overall ACC looks acceptable — pruning or community context may suppress creative generation. Not yet understood.
 - **Medical vs Novel gap:** consistent ~ 0.10 gap (Med ~ 0.73 , Nov ~ 0.63) across all configurations. Entity-centric signals may systematically advantage Medical (denser entity linking) over Novel (broader cultural knowledge).
 - **Judge calibration:** certain questions fail the `calc_fact` judge on every run due to structured JSON parse failures. These are excluded from scoring; impact on reported scores is small but systematic.
 - **Static index:** community detection and entity index are built once. Corpus updates require rebuilding the co-occurrence matrix and community graph — a meaningful constraint for frequently-updated corpora.
-

8 Conclusion

GraphRAG systems outperform vanilla RAG by encoding three structural signals: entity connectivity (P1), community membership (P2), and traversal depth (P3). The standard assumption is that LLM-based graph extraction is necessary to capture these signals at useful fidelity. We show it is not. An implicit graph — NER co-occurrence edges, Louvain community partition — captures the same signals, stacks superadditively across orthogonal retrieval failure modes, and produces better retrieval quality than the best explicit GraphRAG system on GraphRAG-Bench.

The full stack achieves All=0.691 **[VERIFY]** on GraphRAG-Bench, exceeding G-reasoner (0.661) at zero index-time LLM cost and an 8-minute **[VERIFY]** index build versus hours for LLM-based KG construction. The result is not that graphs are unnecessary — it is that LLM extraction is unnecessary to build a graph that works.

The broader implication returns to the motivating agent: re-prompting is a symptom of retrieval incompleteness. Each re-prompting cycle is the agent reconstructing, at inference time, the graph structure that a better retrieval layer would have pre-computed. Cheap NLP primitives are sufficient to pre-compute it.

A Full Grid Results

[VERIFY: insert complete sorted results table after all runs complete]

B Statistical Significance

[VERIFY: bootstrap 95% CIs for all key pairwise comparisons in RQ2–RQ3. Flag deltas below ± 0.015 as below minimum detectable difference at $n=300$, $\alpha=0.05$.]

C Experimental Run Reference

Script	Runs	Status
run_bc_grid_missing.sh	6	✓ complete
run_ablation_structural.sh	2	✓ complete
run_grid_gaps.sh	2	running
run_new_ideas.sh	6	queued
run_paper_validation.sh	4	queued
run_extended_grid.sh	8	queued